

CPSC 304 Project Cover Page

Milestone #: ____2____

Date: ____Feb 25th, 2026_____

Group Number: ____90_____

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Kanaad Kalyan Bodkhe	57135345	b1x5s	bodkhekanaad@gmail.com
Saman Atiq	41874207	r0s7p	samanatiq004@gmail.com
Medhansh Choubey	50146505	d4e7x	medhanshchoubey619@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above.

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

2. Project Summary

This project designs a relational database for managing a law firm's daily operations, including lawyers (across three specializations: family, criminal, and corporate) and their clients, legal cases, work logs, hearings, evidence, and billing. Each case is tied to one client and one primary lawyer, with work tracked over time and invoices generated after case completion. The model also represents hearings held at courthouses and manages evidence at the case level with links to hearings when presented.

3. Programming Language and Database

Database: Oracle (department-provided)

Programming Language/Framework: Java/Spring Boot

4. ER Diagram

Note on ER Diagram Changes: As our group received 18/18 on Milestone 1, there were no mandated changes or suggestions provided by our TA. Therefore, our relational schema is derived directly from our original, unmodified Milestone 1 ER diagram.

5. Relational Schema (Before Normalization)

Client (client_id: INTEGER, name: VARCHAR2(100), contact: VARCHAR2(100), amount_payable: DECIMAL(10,2))

- PK: client_id
- Constraints: name NOT NULL, contact NOT NULL

Lawyer (lawyer_id: INTEGER, bar_id: VARCHAR2(50), name: VARCHAR2(100), start_date: DATE, new_cases: INTEGER, success_rate: DECIMAL(5,2))

- PK: lawyer_id
- CK: bar_id
- Constraints: bar_id UNIQUE NOT NULL, name NOT NULL

Family (lawyer_id: INTEGER, rate: DECIMAL(10,2), mediation_certified: NUMBER(1))

- PK: lawyer_id
- FK: lawyer_id references Lawyer(lawyer_id) ON DELETE CASCADE

Criminal (lawyer_id: INTEGER, rate: DECIMAL(10,2), security_clearance: VARCHAR2(50))

- PK: lawyer_id
- FK: lawyer_id references Lawyer(lawyer_id) ON DELETE CASCADE

Corporate (lawyer_id: INTEGER, rate: DECIMAL(10,2), retainer_required: NUMBER(1))

- PK: lawyer_id
- FK: lawyer_id references Lawyer(lawyer_id) ON DELETE CASCADE

Case (case_id: INTEGER, opening_date: DATE, closing_date: DATE, type: VARCHAR2(50), successful: NUMBER(1), client_id: INTEGER, lawyer_id: INTEGER)

- PK: case_id
- FK 1: client_id references Client(client_id)
- FK 2: lawyer_id references Lawyer(lawyer_id)
- Constraints: client_id NOT NULL (total participation with Client), lawyer_id NOT NULL (total participation with Lawyer), opening_date NOT NULL

WorkLog (lawyer_id: INTEGER, case_id: INTEGER, date: DATE, number_of_hours: DECIMAL(5,2))

- PK: lawyer_id, case_id, date
- FK 1: lawyer_id references Lawyer(lawyer_id)
- FK 2: case_id references Case(case_id)
- Constraints: lawyer_id NOT NULL, case_id NOT NULL (total participation with both)
- Note: date was added to the PK beyond the ER-specified {lawyer_id, case_id} to allow a lawyer to log hours on different days for the same case without violating key constraints.

Invoice (invoice_id: INTEGER, amount: DECIMAL(10,2), status_description: VARCHAR2(20), case_id: INTEGER, status_code: NUMBER(1))

- PK: invoice_id
- CK: case_id
- FK: case_id references Case(case_id)
- Constraints: case_id UNIQUE NOT NULL (1-to-1 with Case, Invoice has total participation), amount NOT NULL
- Note: status_code and status_description were intentionally added to the original Invoice table to introduce a meaningful non-PK functional dependency for the normalization exercise, as required by Milestone 2.

CourtHouse (courtid: INTEGER, judge_name: VARCHAR2(100), city: VARCHAR2(50), province: VARCHAR2(50))

- PK: courtid
- Constraints: city NOT NULL, province NOT NULL

Hearing (case_id: INTEGER, hearing_date: DATE, courtid: INTEGER)

- PK: case_id, hearing_date
- FK 1: case_id references Case(case_id) ON DELETE CASCADE
- FK 2: courtid references CourtHouse(courtid)
- Constraints: courtid NOT NULL (total participation with CourtHouse)

Evidence (evidence_id: INTEGER, stored_in: VARCHAR2(50), type: VARCHAR2(50), case_id: INTEGER)

- PK: evidence_id
- FK: case_id references Case(case_id)
- Constraints: case_id NOT NULL (total participation with Case)

Presented (evidence_id: INTEGER, case_id: INTEGER, hearing_date: DATE)

- PK: evidence_id, case_id, hearing_date
- FK 1: evidence_id references Evidence(evidence_id)
- FK 2: (case_id, hearing_date) references Hearing(case_id, hearing_date)

6. Functional Dependencies (Before Normalization)

Client (client_id, name, contact, amount_payable)

- $\text{client_id} \rightarrow \text{name, contact, amount_payable}$

Lawyer (lawyer_id, bar_id, name, start_date, new_cases, success_rate)

- $\text{lawyer_id} \rightarrow \text{bar_id, name, start_date, new_cases, success_rate}$
- $\text{bar_id} \rightarrow \text{lawyer_id, name, start_date, new_cases, success_rate}$

Family (lawyer_id, rate, mediation_certified)

- $\text{lawyer_id} \rightarrow \text{rate, mediation_certified}$

Criminal (lawyer_id, rate, security_clearance)

- $\text{lawyer_id} \rightarrow \text{rate, security_clearance}$

Corporate (lawyer_id, rate, retainer_required)

- $\text{lawyer_id} \rightarrow \text{rate, retainer_required}$

Case (case_id, opening_date, closing_date, type, successful, client_id, lawyer_id)

- $\text{case_id} \rightarrow \text{opening_date, closing_date, type, successful, client_id, lawyer_id}$

WorkLog (lawyer_id, case_id, date, number_of_hours)

- $(\text{lawyer_id, case_id, date}) \rightarrow \text{number_of_hours}$

Invoice (invoice_id, amount, status_description, case_id, status_code)

- $\text{invoice_id} \rightarrow \text{amount, status_description, case_id, status_code}$
- $\text{case_id} \rightarrow \text{invoice_id, amount, status_description, status_code}$
- $\text{status_code} \rightarrow \text{status_description}$

CourtHouse (courtid, judge_name, city, province)

- courtid $\rightarrow$ judge_name, city, province

Hearing (case_id, hearing_date, courtid)

- (case_id, hearing_date) $\rightarrow$ courtid

Evidence (evidence_id, stored_in, type, case_id)

- evidence_id $\rightarrow$ stored_in, type, case_id

Presented (evidence_id, case_id, hearing_date)

- No non-key attributes; no non-trivial FDs beyond the PK.

7. Normalization

All original ER-derived relations are in BCNF under their listed FDs, with one exception requiring decomposition: **Invoice** (due to status_code $\rightarrow$ status_description).

Invoice Decomposition

Original relation: Invoice(invoice_id, amount, status_description, case_id, status_code)

FDs:

- invoice_id $\rightarrow$ amount, status_description, case_id, status_code
- case_id $\rightarrow$ invoice_id, amount, status_description, status_code
- status_code $\rightarrow$ status_description

Violation: status_code $\rightarrow$ status_description, but status_code is not a superkey of Invoice. This is a transitive dependency (invoice_id $\rightarrow$ status_code $\rightarrow$ status_description), violating 3NF and BCNF.

Decomposition: Remove status_code $\rightarrow$ status_description by splitting off:

Invoice_status (status_code: NUMBER(1), status_description: VARCHAR2(20))

- PK: status_code

Invoice (invoice_id: INTEGER, amount: DECIMAL(10,2), case_id: INTEGER, status_code: NUMBER(1))

- PK: invoice_id
- CK: case_id
- FK 1: case_id references Case(case_id)
- FK 2: status_code references Invoice_status(status_code)
- Constraints: case_id UNIQUE NOT NULL, amount NOT NULL, status_code NOT NULL

Both resulting relations are in BCNF. Every determinant in Invoice (invoice_id, case_id) is a superkey. In Invoice_status, status_code is the PK. The decomposition is lossless since status_code is retained in Invoice as a FK.

Relational Schema After Normalization (All Tables)

Client (client_id: INTEGER, name: VARCHAR2(100), contact: VARCHAR2(100), amount_payable: DECIMAL(10,2))

- PK: client_id
- Constraints: name NOT NULL, contact NOT NULL

Lawyer (lawyer_id: INTEGER, bar_id: VARCHAR2(50), name: VARCHAR2(100), start_date: DATE, new_cases: INTEGER, success_rate: DECIMAL(5,2))

- PK: lawyer_id
- CK: bar_id
- Constraints: bar_id UNIQUE NOT NULL, name NOT NULL

Family (lawyer_id: INTEGER, rate: DECIMAL(10,2), mediation_certified: NUMBER(1))

- PK: lawyer_id
- FK: lawyer_id references Lawyer(lawyer_id) ON DELETE CASCADE

Criminal (lawyer_id: INTEGER, rate: DECIMAL(10,2), security_clearance: VARCHAR2(50))

- PK: lawyer_id
- FK: lawyer_id references Lawyer(lawyer_id) ON DELETE CASCADE

Corporate (lawyer_id: INTEGER, rate: DECIMAL(10,2), retainer_required: NUMBER(1))

- PK: lawyer_id
- FK: lawyer_id references Lawyer(lawyer_id) ON DELETE CASCADE

Case (case_id: INTEGER, opening_date: DATE, closing_date: DATE, type: VARCHAR2(50), successful: NUMBER(1), client_id: INTEGER, lawyer_id: INTEGER)

- PK: case_id
- FK 1: client_id references Client(client_id)
- FK 2: lawyer_id references Lawyer(lawyer_id)
- Constraints: client_id NOT NULL, lawyer_id NOT NULL, opening_date NOT NULL

WorkLog (lawyer_id: INTEGER, case_id: INTEGER, date: DATE, number_of_hours: DECIMAL(5,2))

- PK: lawyer_id, case_id, date
- FK 1: lawyer_id references Lawyer(lawyer_id)
- FK 2: case_id references Case(case_id)
- Constraints: lawyer_id NOT NULL, case_id NOT NULL

Invoice_status (status_code: NUMBER(1), status_description: VARCHAR2(20))

- PK: status_code

Invoice (invoice_id: INTEGER, amount: DECIMAL(10,2), case_id: INTEGER, status_code: NUMBER(1))

- PK: invoice_id
- CK: case_id
- FK 1: case_id references Case(case_id)
- FK 2: status_code references Invoice_status(status_code)
- Constraints: case_id UNIQUE NOT NULL, amount NOT NULL, status_code NOT NULL

CourtHouse (courtid: INTEGER, judge_name: VARCHAR2(100), city: VARCHAR2(50) province: VARCHAR2(50))

- PK: courtid
- Constraints: city NOT NULL, province NOT NULL

Hearing (case_id: INTEGER, hearing_date: DATE, courtid: INTEGER)

- PK: case_id, hearing_date
- FK 1: case_id references Court_Case(case_id) ON DELETE CASCADE
- FK 2: courtid references CourtHouse(courtid)
- Constraints: courtid NOT NULL

Evidence (evidence_id: INTEGER, stored_in: VARCHAR2(50), type: VARCHAR2(50), case_id: INTEGER)

- PK: evidence_id
- FK: case_id references Case(case_id)
- Constraints: case_id NOT NULL

Presented (evidence_id: INTEGER, case_id: INTEGER, hearing_date: DATE)

- PK: evidence_id, case_id, hearing_date
- FK 1: evidence_id references Evidence(evidence_id)
- FK 2: (case_id, hearing_date) references Hearing(case_id, hearing_date)

Functional Dependencies After Normalization

Client: client_id → name, contact, amount_payable

Lawyer: lawyer_id → bar_id, name, start_date, new_cases, success_rate | bar_id → lawyer_id, name, start_date, new_cases, success_rate

Family: lawyer_id → rate, mediation_certified

Criminal: lawyer_id → rate, security_clearance

Corporate: lawyer_id → rate, retainer_required

Case: case_id → opening_date, closing_date, type, successful, client_id, lawyer_id

WorkLog: (lawyer_id, case_id, date) → number_of_hours

Invoice_status: status_code → status_description

Invoice: invoice_id → amount, status_code, case_id | case_id → invoice_id, amount, status_code

CourtHouse: courtid → judge_name, city, province

Hearing: (case_id, hearing_date) → courtid

Evidence: evidence_id → stored_in, type, case_id

Presented: No non-key attributes.

8. SQL DDL

Implementation Notes

1. Foreign Key Handling: As mentioned in class, Oracle does not support ON UPDATE CASCADE. For our project, we have implemented ON DELETE CASCADE where required by our ERD. We recognize that ON UPDATE CASCADE would typically be used for identifying relationships, but due to Oracle's limitations, this must be handled differently.

2. Naming Consistency: We intentionally changed several names from our ER diagram to comply with Oracle SQL reserved keywords. Specifically, 'Case' was changed to 'Court_Case', 'successful' to 'is_successful', 'type' to 'case_type' and 'evidence_type', and 'date' to 'log_date'.

3. Unmodellable Constraints: We understand that certain participation constraints cannot be fully expressed using basic SQL DDL right now. We will enforce these constraints at a later point in the project.

```
-- Parent Tables

CREATE TABLE Client (

    client_id INTEGER PRIMARY KEY,

    name VARCHAR2(100) NOT NULL,

    contact VARCHAR2(100) NOT NULL,

    amount_payable DECIMAL(10,2)

);

CREATE TABLE Lawyer (

    lawyer_id INTEGER PRIMARY KEY,

    bar_id VARCHAR2(50) UNIQUE NOT NULL,

    name VARCHAR2(100) NOT NULL,

    start_date DATE,

    new_cases INTEGER,
```

```
        success_rate DECIMAL(5,2)
    );

CREATE TABLE CourtHouse (

    courtid INTEGER PRIMARY KEY,

    judge_name VARCHAR2(100),

    city VARCHAR2(50) NOT NULL,

    province VARCHAR2(50) NOT NULL

);

CREATE TABLE Invoice_status (

    status_code NUMBER(1) PRIMARY KEY,

    status_description VARCHAR2(20)

);

-- Child Tables - Level 1 Dependencies

CREATE TABLE Family (

    lawyer_id INTEGER PRIMARY KEY,

    rate DECIMAL(10,2),

    mediation_certified NUMBER(1),

    FOREIGN KEY (lawyer_id) REFERENCES Lawyer(lawyer_id) ON DELETE CASCADE

);
```

```
CREATE TABLE Criminal (  
  
    lawyer_id INTEGER PRIMARY KEY,  
  
    rate DECIMAL(10,2),  
  
    security_clearance VARCHAR2(50),  
  
    FOREIGN KEY (lawyer_id) REFERENCES Lawyer(lawyer_id) ON DELETE CASCADE  
  
);
```

```
CREATE TABLE Corporate (  
  
    lawyer_id INTEGER PRIMARY KEY,  
  
    rate DECIMAL(10,2),  
  
    retainer_required NUMBER(1),  
  
    FOREIGN KEY (lawyer_id) REFERENCES Lawyer(lawyer_id) ON DELETE CASCADE  
  
);
```

```
CREATE TABLE Court_Case (  
  
    case_id INTEGER PRIMARY KEY,  
  
    opening_date DATE NOT NULL,  
  
    closing_date DATE,  
  
    case_type VARCHAR2(50),  
  
    is_successful NUMBER(1),  
  
    client_id INTEGER NOT NULL,  
  
    lawyer_id INTEGER NOT NULL,  
  
    FOREIGN KEY (client_id) REFERENCES Client(client_id),  
  
    FOREIGN KEY (lawyer_id) REFERENCES Lawyer(lawyer_id)
```

```

);

-- Child Tables - Level 2 Dependencies

CREATE TABLE WorkLog (

    lawyer_id INTEGER NOT NULL,

    case_id INTEGER NOT NULL,

    log_date DATE,

    number_of_hours DECIMAL(5,2),

    PRIMARY KEY (lawyer_id, case_id, log_date),

    FOREIGN KEY (lawyer_id) REFERENCES Lawyer(lawyer_id),

    FOREIGN KEY (case_id) REFERENCES Court_Case(case_id)

);

CREATE TABLE Invoice (

    invoice_id INTEGER PRIMARY KEY,

    amount DECIMAL(10,2) NOT NULL,

    case_id INTEGER UNIQUE NOT NULL,

    status_code NUMBER(1) NOT NULL,

    FOREIGN KEY (case_id) REFERENCES Court_Case(case_id),

    FOREIGN KEY (status_code) REFERENCES Invoice_status(status_code)

);

```

```
CREATE TABLE Hearing (  
  
    case_id INTEGER,  
  
    hearing_date DATE,  
  
    courtid INTEGER NOT NULL,  
  
    PRIMARY KEY (case_id, hearing_date),  
  
    FOREIGN KEY (case_id) REFERENCES Court_Case(case_id) ON DELETE CASCADE,  
  
    FOREIGN KEY (courtid) REFERENCES CourtHouse(courtid)  
  
);
```

```
CREATE TABLE Evidence (  
  
    evidence_id INTEGER PRIMARY KEY,  
  
    stored_in VARCHAR2(50),  
  
    evidence_type VARCHAR2(50),  
  
    case_id INTEGER NOT NULL,  
  
    FOREIGN KEY (case_id) REFERENCES Court_Case(case_id)  
  
);
```

```
-- Child Tables - Level 3 Dependencies

CREATE TABLE Presented (

    evidence_id INTEGER,

    case_id INTEGER,

    hearing_date DATE,

    PRIMARY KEY (evidence_id, case_id, hearing_date),

    FOREIGN KEY (evidence_id) REFERENCES Evidence(evidence_id),

    FOREIGN KEY (case_id, hearing_date) REFERENCES Hearing(case_id, hearing_date)

);
```

8b. Foreign Key Handling Discussion

Family / Criminal / Corporate → Lawyer (ON DELETE CASCADE): A specialization record has no meaning without its parent Lawyer row. Deleting a lawyer should remove their specialization data automatically. ON UPDATE CASCADE would be appropriate here but Oracle does not support it.

Court_Case → Client (no action): Deleting a client should not silently erase case history. This should be handled explicitly at the application level.

Court_Case → Lawyer (no action): Same reasoning — case records are historical data and should not be cascade-deleted when a lawyer is removed.

WorkLog → Lawyer / Court_Case (no action): Work log entries are billing and audit records. They should be preserved even if the associated lawyer or case record is deleted.

Invoice → Court_Case (no action): An invoice is a financial record. Its existence should be preserved independently of changes to the case.

Invoice → Invoice_status (no action): Invoice_status is a lookup table. Attempting to delete a status code that active invoices reference will be blocked by Oracle's default referential integrity behavior, which is the correct outcome.

Hearing → Court_Case (ON DELETE CASCADE): Hearing is an identifying (weak) entity of Court_Case. A hearing has no independent meaning without its associated case, so cascade deletion is appropriate.

Hearing → Courthouse (no action): Deleting a courthouse should not erase historical hearing records. Reassignment or archival should be handled at the application level.

Evidence → Court_Case (no action): Evidence is critical legal data and must not be deleted as a side effect of case deletion.

Presented → Evidence / Hearing (no action): Presented records are historical and should be preserved independently.

9. AI Declaration

We used ChatGPT to help us with this assignment. The images of the conversation are attached below.

Project Description:

Domain of the application:

The domain of our application is law firm case management. It represents the day-to-day operational aspects of a legal practice, including the cases a lawyer is handling, the work performed over time, and the progression of cases through hearings and courthouses. It also encompasses the tracking of evidence associated with each case and hearing, as well as the billing process upon case completion.

What aspects of the domain are modeled by the database.

Client and Case Structure

We store information about clients and the cases they bring to the firm. Each case is associated with exactly one client, as every case must belong to a lawyer through that client relationship. A client may exist in the system even if they do not currently have any active cases.

Lawyer Assignment and Practice Areas

Each case is assigned one primary lawyer to ensure clear accountability. Lawyers can exist in the system even if they are not currently assigned to any cases. We also model that each lawyer practices in exactly one area—Family, Criminal, or Corporate—so the system can support different billing rates and rules based on the type of law practiced.

Work Logs and Case Effort Tracking

Lawyers record their work through work logs. Each work log is linked to exactly one lawyer and one case, allowing the system to track effort over time and enabling accurate billing once the case is complete.

Billing and Invoicing

Billing occurs only after a case has been completed. As a result, an open case may not have an invoice, but once it is closed, it can generate a final invoice. This reflects common law firm billing practices while keeping the data model straightforward and realistic.

Hearings and Courthouses

Hearings are dependent on cases, as a hearing cannot exist without an associated case. For this reason, hearings are treated as weak entities, identified by the case and the hearing date. Each hearing must take place at exactly one courthouse, since a hearing requires a physical location.

Evidence Management

Evidence is associated with a case rather than directly with a hearing, because evidence is gathered for the overall case. Evidence may later be presented at one or more hearings, or not presented at all. This is why evidence and hearings are modeled as a many-to-many relationship.

The system models core law firm operations by capturing clients, cases, lawyers, and their relationships. Each case belongs to a single client and is managed by one primary lawyer, who practices in a specific area of law. Lawyers track their work through work logs linked to cases, enabling accurate billing once a case is closed. The model also represents hearings as case-dependent events held at courthouses and manages evidence at the case level, allowing it to be associated with multiple hearings or none at all.

this is what our project entails. now we are doing milestone 2 where the professor wants us to provide a short project summary of the project. summarize our project into a bried 2-3 line summary that will let the TA checking our assignment know what our project is about. don't use words from elsewhere, just use only this information to form a summary.

This project designs a database for managing a law firm's daily operations, including clients, cases, lawyers, work logs, hearings, evidence, and billing. Each case is tied to one client and one primary lawyer, with work tracked over time and invoices generated after case completion. The model also represents hearings held at courthouses and manages evidence at the case level with links to hearings when presented.



instead of just saying database, say relational database.

This project designs a relational database for managing a law firm's daily operations, including clients, cases, lawyers, work logs, hearings, evidence, and billing. Each case is tied to one client and one primary lawyer, with work tracked over time and invoices generated after case completion. The model also represents hearings held at courthouses and manages evidence at the case level with links to hearings when presented.



for Oracle, does it support ON UPDATE CASCADE?

Thought for 13s >

No. Oracle Database does not support `ON UPDATE CASCADE` in foreign key constraints, the `ON UPDATE` clause is not valid in Oracle SQL. [asktom.oracle.com](#) +1



so for making relational schema about our project should i use `ON DELETE CASCADE` then? and make a note about in our project? For milestone 2, they want us to convert the ER diagram into the relational schema, normalize it and then write the SQL DDL code for creating the tables for it. we are thinking of using this: Database: Oracle (department-provided)
Programming Language/Framework: Java/Spring Boot

Thought for 14s >

Do not add `ON DELETE CASCADE` everywhere just because Oracle lacks `ON UPDATE CASCADE`.

Oracle supports `ON DELETE CASCADE` and `ON DELETE SET NULL` on foreign keys, but it does not implement `ON UPDATE CASCADE`. [TechOnTheNet +1](#)

Use `ON DELETE CASCADE` only where the child row has no independent meaning without the parent, typical in your schema:

- Weak or dependent tables: `Hearing` depends on `Case` (and is identified by `case + hearing_date`), so `Hearing.case_id -> Case.case_id ON DELETE CASCADE` is reasonable.
- Pure associative tables: the evidence to hearing link table (many to many) should cascade from both parents so the link rows do not become orphaned.

Avoid `ON DELETE CASCADE` on core, auditable business data (common in case management):

- `Case -> Client`, `Case -> Lawyer`, `WorkLog -> Case/Lawyer`, `Invoice -> Case` are usually better as `RESTRICT`, meaning no cascade, because deleting a client, case, lawyer, or billed work typically should be blocked or handled via a status field, not

ChatGPT 5.2

✎ ↗ ...

milestone note to include (its Oracle + your design).

- "Oracle foreign keys support `ON DELETE` actions (`CASCADE/SET NULL`) but not `ON UPDATE CASCADE`, so primary keys are treated as immutable identifiers and the schema avoids update cascading." [Oracle Forums +1](#)

📄 👍 💬 ↗ ↻ ...

what is the difference between `VARCHAR`, `VARCHAR2`, and `CHAR`

`CHAR`

- Fixed length.
- If you define `CHAR(10)` and store "Law", Oracle stores "Law " padded with spaces to length 10.
- Always consumes the full declared length.
- Good only when values are truly fixed length, e.g., 2-letter province codes.

VARCHAR2

- Variable length.
- `VARCHAR2(10)` stores only the actual characters entered, up to 10.
- No automatic padding.
- Recommended string type in Oracle.
- Maximum size depends on database configuration, typically up to 4000 bytes in standard mode.

VARCHAR

- Currently behaves the same as `VARCHAR2` in Oracle.
- Reserved for future use by Oracle.
- Oracle documentation recommends using `VARCHAR2`, not `VARCHAR`, to avoid potential future behavior changes.

Practical Rule for Your Project

For your relational schema in Oracle:

- Use `VARCHAR2` for names, descriptions, emails, case titles, evidence notes.
- Use `CHAR` only for truly fixed-length codes.
- Avoid `VARCHAR`.

